

Real-Time Systems - Laboratory Exercise 3 (week 4)

Lab Note: These tasks are to be conducted individually (but you should discuss with your friend, don't get lost alone). Although you don't need to submit a report for assessment as in Melbourne, you should make a **note document** and keep your solution/s **code (*.c file/s)**, so that you can review later when doing the final project, as well as prepare for the In-class Tests.

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. Finally, keep the source files (*.c files) together with the note.

Mutual Exclusion and Data Protection

In lab exercises 2, the concept of POSIX message queues was explored, which allows messages to be sent and received between processes (even processes running on different machines). However, message queues involve system overhead including the time required to create or open the message queue, and to make a copy of the message when sending it (the copy is stored in the queue), and to make another copy of the message when receiving it. This overhead is usually acceptable when passing messages between processes (and unavoidable when sending messages to processes on another node) as messages are often non-trivial and may only be sent occasionally.

However, processes running on the same node often want to share data frequently (e.g. boolean flags) or send small messages (e.g. single variables) to each other, and the overhead of message queues may be considered excessive.

Given that threads in the same process share the same memory space, it therefore makes sense that they can access and/or modify any global variables with almost no overhead. However, doing so may lead to safety problems, namely that the variables may not end up with the "correct" or "expected" value (as explained in the lecture material on Concurrent Processes).

The <pthread.h> library includes various functions which, if used correctly, can ensure safety and correct operation of threads when sharing access to the same memory space (i.e. variables).

You should also download the [overlap1.c](#) file from Canvas and create a new Momentics project.

Follow below guide, then compile and execute each example/code to verify behavior of each data protection example. Make sure that you understand the operation and usage of mutexes and Reader/Writer Locks. *Note the use of usleep(1) in the threads to ensure all threads will run concurrently – simulating a multi-threaded application. If you find that one thread seems to hog the CPU adding this should fix things, but as you will see, you will need to consider data protection principles in your design to achieve the correct program behaviour.* You should end up with 4 or more different programs within your workspace as you work through the below examples. Make sure you run your code samples on both single CPU targets and multi-CPU targets to explore the behavior on quasi-concurrent and truly concurrent targets.

Protecting Your Data in a Multi-threaded App by John Fehr, QNX Software Systems Ltd.

The most important part of your application is most likely the data you're manipulating. If, for some reason, that data is corrupted, your application is useless. It's pretty straightforward to make sure your single-threaded application doesn't accidentally corrupt its data. Multi-threaded applications are quite another story, however.

Task 1A: The overlap problem

The problem with multi-threaded applications is that you could have one section of code modifying your data, while another section is reading and using that same data. If the write and read overlap, you could have some serious complications!

Why don't you try creating a little application and some data to be modified?

```
#include <stdio.h>
#include <pthread.h>
```

```
typedef struct
{
    int a;
    int b;
    int result;
    int result2;
    int use_count;
    int use_count2;
    int max_use;
    int max_use2;
} app_data;
```

Pass your app_data structure to a thread that reads the data and uses the a and b variables 100 times whenever a is set to five. You'll use a local "uses" variable to count 100 of your uses. You'll also increment the application's "use_count" in case someone else wants to keep track of how many times your data has been used. You'll add a small usleep here to make sure the scheduler gives other threads a chance to run. Here's the code for your data user:

```
void *user_thread(void *data)
{
    int uses=0;
    app_data *td=(app_data*)data;

    while(uses<td->max_use)
    {
        if (td->a==5)
        {
            td->result+=(td->a+td->b);
            td->use_count++;
            uses++;
        }
        usleep(1);
    }

    return 0;
}
```

Now, make some code that changes your data. You'll want to continue changing a and b until they've been used elsewhere (your user_thread, in this case) 100 times. Your change will be simple: you'll simply toggle your a value between 5 and 50 and fake a CPU-intensive calculation for b with a **usleep(1000)** call. (Usleep will return 0 unless there's something really wrong with your machine.) This means that a will be changed and 1 ms later b will be changed. You'll have a 1 ms gap where a and b should not be used elsewhere in the application since they haven't been set to what you want yet:

```
void *changer_thread(void *data)
{
    app_data *td=(app_data*)data;

    while ((td->use_count+td->use_count2)<(td->max_use+td->max_use2))
    {
        if (td->a==5)
        {
            td->a=50;
            td->b=td->a+usleep(1000);
        }
        else
        {
            td->a=5;
            td->b=td->a+usleep(1000);
        }
        usleep(1);
    }

    return 0;
}
```

Wrap it all up in your main() by creating the app_data instance, spawning your two threads, waiting for them to finish, and printing the results. They should be:

```
main()
{
    pthread_t ct,ut,st;
    app_data td={5,5,0,0,0,0,100,0};
    void *retval;

    pthread_create(&ut,NULL,user_thread,&td);
    pthread_create(&ct,NULL,changer_thread,&td);

    pthread_join(ct,&retval);
    pthread_join(ut,&retval);
    printf("result should be %d, is %d\n",td.max_use*(5+5),td.result);
}
```

Now build this puppy (the code for this example is in overlap1.c) and see how she runs:

```
$ gcc overlap1.c -o overlap1
$ ./overlap1
result should be 1000, is 3967
```

Your result might be different depending on your CPU speed. Clearly this isn't what you want! Why did you get such a weird result?

The problem is that the **user_thread** is waiting until a is set to 5, and when it is, it adds the a and b values (which should both be 5) to the result variable. However, since the b variable is taking so long to get calculated, it sometimes uses the old b value (50) instead of the new one (5). You need to protect your data somehow so that the **user_thread** can't use the data until you've protected it!

Task 1B: Mutexes

Welcome to the wonderful world of the mutex (mutual exclusion). As the name implies, mutexes help you make sure that certain code or data is only accessed by one thread at a time.

Mutexes are very easy to use. Once a mutex has been created, you simply put a **pthread_mutex_lock()** in front of the code you want to protect and **pthread_mutex_unlock()** right after that code. In this case, you want to protect both the modification of the a and b variables as well as their use. First, though, you need to put in code to create and delete the mutex. Put the mutex into this handy structure:

```
typedef struct
{
    int a;
    int b;
    int result;
    int result2;
    int use_count;
    int use_count2;
    int max_use;
    int max_use2;
    pthread_mutex_t mutex;
} app_data;
```

Create and destroy the mutex in your **main()** function. In this example, you'll pass NULL as the attr parameter to use the default attributes for the mutex:

```
main()
{
    pthread_t ct,ut,st;
    app_data td={5,5,0,0,0,0,100,0};
    void *retval;

    pthread_mutex_init(&td.mutex,NULL);
    pthread_create(&ut,NULL,user_thread,&td);
    pthread_create(&ct,NULL,changer_thread,&td);

    pthread_join(ct,&retval);
    pthread_join(ut,&retval);
    pthread_mutex_destroy(&td.mutex);
    printf("result should be %d, is %d\n",td.max_use*(5+5),td.result);
}
```

Now, you put the lock/unlock functions around the code you want to protect (when you're reading or writing the a or b variables). Change the **while()** loop in the user thread to read:

```
while(uses<td->max_use)
{
    pthread_mutex_lock(&td->mutex);
    if (td->a==5)
    {
```

```

    td->result+=(td->a+td->b);
    td->use_count++;
    uses++;
}
pthread_mutex_unlock(&td->mutex);
usleep(1);
}

```

Also, change the changer thread's **while()** loop to read:

```

while ((td->use_count+td->use_count2)<(td->max_use+td->max_use2))
{
    pthread_mutex_lock(&td->mutex);
    if (td->a==5)
    {
        td->a=50;
        td->b=td->a+usleep(1000);
    }
    else
    {
        td->a=5;
        td->b=td->a+usleep(1000);
    }
    pthread_mutex_unlock(&td->mutex);
    usleep(1);
}

```

This time when you compile (the code for this version is in overlap2.c) and run it you get: result should be 1000, but is 1000 which is exactly the result you want. :)

Task 1C: Multiple Readers

What if you have one thread that's changing your assets, but more than one other thread that just wants to read your assets?

Add another thread that needs to read a and b as well:

```

void *subtractor_thread(void *data)
{
    int use=0;
    app_data *td=(app_data*)data;

    while(use<td->max_use2)
    {
        pthread_mutex_lock(&td->mutex);
        if (td->a==50)
        {
            td->result2-=(td->a+td->b);
            use++;
            td->use_count2++;
        }
        pthread_mutex_unlock(&td->mutex);
        usleep(1);
    }
}

```

```
    return 0;
}
```

You'll also need to modify **main()** to spawn the new thread and set your **max_use2** variable in your **app_data** structure:

```
main()
{
    pthread_t ct,ut,st;
    app_data td={5,5,0,0,0,0,100,100};
    void *retval;

    pthread_mutex_init(&td.mutex,NULL);
    pthread_create(&ut,NULL,user_thread,&td);
    pthread_create(&ct,NULL,changer_thread,&td);
    pthread_create(&st,NULL,subtractor_thread,&td);

    pthread_join(st,&retval);
    pthread_join(ct,&retval);
    pthread_join(ut,&retval);
    pthread_mutex_destroy(&td.mutex);
    printf("result should be %d, is %d\n",td.max_use*(5+5),td.result);
    printf("result2 should be %d, is %d\n",-(td.max_use2*(50+50)),td.result2);
}
```

If you compile and run this (the code for this version is in **overlap3.c**), you get just what you wanted:

```
result should be 1000, is 1000
result2 should be -10000, is -10000
```

But wait. Shouldn't the **user_thread** and the **subtractor_thread** read **a** and **b** at the same time? (Remember that with mutexes anything wrapped in the lock/unlock pair can't be executed at the same time as another piece of code wrapped in the lock/unlock pair.) Neither of them change **a** or **b**, so it would be nice if you could let both of them read **a** and **b** at the same time - just not when the **changer_thread** is changing them.

Task 1D: Rwlocks

With **rwlocks**, another locking mechanism, you can easily implement this kind of behavior. Unlike mutexes, rwlocks can be locked either as read or as write. As long as they're not locked for write access, any threads can lock for read and unlock as much as they want. However, once they're locked for write access, all read locks occurring afterwards are blocked until the write lock is unlocked. Sounds perfect, eh?

First you'll need to change your mutex to a rwlock in your structure:

```
typedef struct
{
    int a;
    int b;
    int result;
    int result2;
    int use_count;
    int use_count2;
```

```

int max_use;
int max_use2;
pthread_rwlock_t rwl;
} app_data;

```

Next, modify your `main()` so that the `rwlock` is initialized instead of a `mutex`. You use `NULL` again as the `attr` argument to get the default behavior:

```

main()
{
    pthread_t ct,ut,st;
    app_data td={5,5,0,0,0,0,100,100};
    void *retval;

    pthread_rwlock_init(&td.rwl);

    pthread_create(&ut,NULL,user_thread,&td);
    pthread_create(&ct,NULL,changer_thread,&td);
    pthread_create(&st,NULL,subtractor_thread,&td);

    pthread_join(st,&retval);
    pthread_join(ct,&retval);
    pthread_join(ut,&retval);
    pthread_rwlock_destroy(&td.rwl);
    printf("result should be %d, is %d\n",td.max_use*(5+5),td.result);
    printf("result2 should be %d, is %d\n",-(td.max_use2*(50+50)),td.result2);
}

```

Finally, you change the **`pthread_mutex_lock`** to **`pthread_rwlock_rdlock`** and the **`pthread_mutex_unlock`** to **`pthread_rwlock_unlock`** in your **`user_thread`** and **`changer_thread`**. That takes care of the read access. You also replace the **`pthread_mutex_lock`** to **`pthread_rwlock_wrlock`** and the **`pthread_mutex_unlock`** to **`pthread_rwlock_unlock`** in your **`changer_thread`** function, to take care of the write access. Here's your new code:

```

void *user_thread(void *data)
{
    int uses=0;
    app_data *td=(app_data*)data;

    while(uses<td->max_use)
    {
        pthread_rwlock_rdlock(&td->rwl);
        if (td->a==5)
        {
            td->result+=(td->a+td->b);
            td->use_count++;
            uses++;
        }
        pthread_rwlock_unlock(&td->rwl);
        usleep(1);
    }
}

```

```
    return 0;
}

void *changer_thread(void *data)
{
    app_data *td=(app_data*)data;

    while ((td->use_count+td->use_count2)<(td->max_use+td->max_use2))
    {
        pthread_rwlock_wrlock(&td->rw1);
        if (td->a==5)
        {
            td->a=50;
            td->b=td->a+usleep(1000);
        }
        else
        {
            td->a=5;
            td->b=td->a+usleep(1000);
        }
        pthread_rwlock_unlock(&td->rw1);
        usleep(1);
    }

    return 0;
}

void *subtractor_thread(void *data)
{
    int use=0;
    app_data *td=(app_data*)data;

    while(use<td->max_use2)
    {
        pthread_rwlock_rdlock(&td->rw1);
        if (td->a==50)
        {
            td->result2-=(td->a+td->b);
            use++;
            td->use_count2++;
        }
        pthread_rwlock_unlock(&td->rw1);
        usleep(1);
    }

    return 0;
}
```

That's it! (The code for this version is in overlap4.c)

Now the **subtractor_thread** and the **user_thread** can read the data at the same time, but the **changer_thread** can't access the data at the same time as either the **subtractor_thread** or the **user_thread**.

One final note on the use of rwlocks: they are not recursive and therefore are not promotive. In other words, you can't lock your rwlock over and over again without unlocking first. You also can't lock your rwlock for read and then for write without unlocking first. This tradeoff was done to keep the rwlock as lightweight (in terms of size and speed) as possible.

I hope I've helped you figure out how, where, and why to use mutexes and rwlocks!